Конспект: Алгоритмы и структуры данных, лекции 2–6

Oct 26, 2026 · @Sokrat

С чего начать

Этот конспект написан для того, кто только начинает программировать. Читай по порядку: каждая лекция опирается на предыдущую. Код в серых блоках взят со слайдов, я только добавил комментарии на русском.

Лекция	О чём одной фразой
2	Как текст на языке C превращается в программу, которую можно запустить
3	Из чего состоит программа на C: функции, переменные, условия, циклы, память
4	Как автоматически проверить, что код работает правильно (тесты)
5	Сколько места в памяти занимают числа, буквы и структуры, и что такое указатель
6	Как сделать массив, который сам растёт, и связный список. Именно их ты будешь писать в лабах

Файлы «л1_2.pdf» и «л1_2 1.pdf» одинаковые, внутри — лекция 2. Отдельной лекции 1 в папке нет.

Словарик: слова, без которых дальше не понять

- **Переменная** — коробка с именем, в которой лежит значение. `int age = 18;` — коробка `age`, в ней число 18.
- **Тип** — что именно можно положить в коробку и какого она размера. `int` — целое число, `float` и `double` — дробное, `char` — один символ, `bool` — да/нет. В C тип нужно писать всегда, в отличие от Python.
- **Функция** — кусок кода с именем. Ей дают данные (аргументы), она что-то делает и возвращает результат. Как кофемашина: засыпал зёрна — получил кофе.

- **Память** — очень длинная лента из ячеек по 1 байту. У каждой ячейки есть номер — **адрес**. Все переменные лежат где-то на этой ленте.
- **Байт** — 8 бит (ноликов и единичек). В один байт влезает число от 0 до 255.
- **Указатель** — переменная, в которой лежит не число, а адрес другой переменной. Как бумажка с адресом дома вместо самого дома.
- **Стек** — память для обычных переменных внутри функции. Они появляются сами и сами исчезают, когда функция заканчивается.
- **Куча** — память, которую ты просишь сам (функцией `malloc`) и сам отдаёшь обратно (функцией `free`). Если забыть отдать, будет **утечка памяти**.
- **Компилятор** — программа, которая переводит твой код на C в команды для процессора. На курсе это `gcc`.
- **Массив** — несколько коробок одного типа, стоящих вплотную друг за другом. `int a[3]` — три целых числа подряд: `a[0]`, `a[1]`, `a[2]`. Счёт всегда начинается с нуля.

Лекция 2. Как код становится программой

Главная мысль: процессор не понимает ни C, ни Python. Он понимает только машинный код — набор ноликов и единиц. Значит, кто-то должен перевести твой текст на этот язык.

Компилятор и интерпретатор

Программа-переводчик называется **транслятор**. Они бывают двух видов, и разницу легко понять на примере с книгой.

- **Компилятор** — как переводчик книги. Он сначала переводит всё целиком и отдаёт готовый файл. Потом ты запускаешь этот файл сколько угодно раз, и переводчик уже не нужен. Так работает C.
- **Интерпретатор** — как синхронный переводчик. Он читает код строчка за строчкой и тут же выполняет. Так работает Python.

Пример со слайда — интерпретатор Python в терминале. Ты пишешь строчку, и она сразу выполняется:

```
% python
>>> print(1)     ← ты написал
1                ← интерпретатор сразу выполнил
```

У Python несколько интерпретаторов. Обычный называется **CPython**. Его ругают за медлительность и за GIL — ограничение, из-за которого код не может по-настоящему работать в несколько потоков. Есть и другие: PyPy (быстрее в 2–5

раз), MicroPython (для микроконтроллеров), Brython (Python в браузере). Запоминать их не обязательно.

Как gcc собирает программу: 4 шага

Разберём на примере со слайда. Эта программа складывает числа от 1 до 10 и печатает `sum = 55`:

```
#include <stdio.h>      // подключаем библиотеку, где есть printf
#define LAST 10         // говорим: везде, где написано LAST, подставь 10

int main()
{
    int sum = 0;                            // сначала сумма равна 0
    for (int i = 1; i <= LAST; i++) {       // i идёт от 1 до 10
        sum += i;                           // добавляем i к сумме
    }
    printf("sum = %d\n", sum);              // печатаем: sum = 55
    return 0;                               // 0 значит «всё хорошо»
}
```

Чтобы превратить этот текст в программу, gcc делает четыре шага по очереди. Обычно ты их не видишь, но команда `gcc -Wall -save-temps test.c -o test` сохранит файл после каждого шага, и можно посмотреть.

1. **Препроцессинг** (файл `.i`). Это просто замена текста, как «Найти и заменить» в Word. Все слова `LAST` меняются на `10`. Вместо `#include <stdio.h>` вставляется весь файл stdio.h. Комментарии удаляются. Результат — всё ещё текст на C.
2. **Компиляция** (файл `.s`). Код переводится на ассемблер — язык простых команд процессора вроде «взять число», «вычесть 10», «если больше нуля, перейти на выход».
3. **Ассемблирование** (файл `.o`). Ассемблер превращается в машинный код. Но запустить этот файл ещё нельзя: в нём есть вызов `printf`, а самого кода `printf` там нет.
4. **Линковка** (готовая программа). Линкер берёт твой `.o`, находит в библиотеках код `printf` и склеивает всё в один файл. Его уже можно запустить.

Если хочешь сделать шаги вручную, по одному:

```
gcc -E main.c -o main.i   # 1. препроцессинг
gcc -S main.i -o main.s   # 2. компиляция в ассемблер
gcc -c main.s -o main.o   # 3. ассемблирование в объектный файл
gcc main.o -o main        # 4. линковка в готовую программу
```

На шаге 2 внутри компилятора происходит ещё кое-что интересное. Сначала текст режется на «слова» (лексемы). Например, `a = 2;` превращается в четыре куска: имя `a`, знак `=`, число `2`, точка с запятой. Потом из них строится дерево (AST): например, для `temp1 + temp2` плюс становится вершиной, а `temp1` и `temp2` — его ветками. Из дерева уже получается ассемблер.

Про библиотеки на шаге 4. Их можно подключить двумя способами. Статически (`.a`) — код библиотеки копируется прямо в твою программу: файл больше, но работает сам по себе. Динамически (`.so` на Linux, `.dylib` на Mac, `.dll` на Windows) — библиотека подгружается при запуске: файл меньше, но библиотека должна быть установлена.

Компиляторы и как запустить программу

Компиляторов C несколько. Тебе достаточно знать три: **GCC** (стандарт на Linux), **Clang** (стоит на Mac, пишет понятные ошибки) и **MSVC** (Windows, Visual Studio). На Mac команда `gcc` на самом деле запускает Clang — это нормально.

Немного истории: язык C придумал Dennis Ritchie в 1972 году, чтобы написать на нём операционную систему UNIX. C++ появился в 1983 как «C с классами».

Как запустить свою первую программу:

```
gcc --version          # проверить, что компилятор есть
gcc test.c -o test     # собрать: из test.c получится файл test
./test                 # запустить (на Mac и Linux)
```

Если компилятора нет: на Mac — `brew install gcc`, на Linux — `sudo apt install build-essential`.

Makefile: чтобы не печатать одно и то же

Команда сборки бывает длинной, например `gcc -Wall -Wextra -O2 test.c -o test`. Набирать её каждый раз скучно, поэтому её записывают один раз в файл `Makefile`. Потом достаточно написать одно слово `make`. Вот Makefile со слайда:

```
TARGET = test                  # как назовётся готовая программа
CC = gcc                       # какой компилятор
CFLAGS = -Wall -Wextra -O2     # флаги: показывать все предупреждения,
оптимизировать
SRC = test.c                   # файл с кодом

all: $(TARGET)                 # что делать по команде "make"

$(TARGET): $(SRC)              # чтобы получить test, нужен test.c
        $(CC) $(CFLAGS) $(SRC) -o $(TARGET)

run: $(TARGET)                 # "make run" — собрать и запустить
        ./$(TARGET)

clean:                         # "make clean" — удалить собранную программу
        rm -f $(TARGET)
```

И что ты увидишь в терминале:

```
% make run
gcc -Wall -Wextra -O2 test.c -o test
./test
sum = 55
```

Важно: строки с командами в Makefile должны начинаться с **табуляции**, а не с пробелов. Иначе make выдаст ошибку. Для больших проектов есть инструменты помощнее (CMake, Ninja, Meson, Bazel), но на курсе хватит Makefile.

Как читать ошибки: три примера со слайдов

Ошибка 1 — в Makefile.

```
make: *** No rule to make target `test1.c', needed by `test'. Stop.
```

Перевод: «чтобы собрать test, мне нужен файл test1.c, а его нет». Значит, в Makefile опечатка в имени файла.

Ошибка 2 — забыли точку с запятой.

```
test.c:6:19: error: expected ';' at end of declaration
    6 | int i, sum = 0
      |               ^
      |               ;
```

`test.c:6:19` — это файл, строка 6, символ 19. Компилятор даже показал галочкой `^`, куда поставить `;`.

Ошибка 3 — переменная вне своей области.

```
test.c:11:24: error: use of undeclared identifier 'i'
   11 |     printf("i = %d\n", i);
```

Перевод: «я не знаю, что такое i». Скорее всего, `i` была объявлена внутри `for (int i = ...)` и после цикла уже не существует. Подробнее про это — в лекции 3, в разделе про область видимости.

Совет: читай всегда **первую** ошибку в списке. Остальные часто из неё же и вытекают. В VS Code поставь расширение C/C++: оно подчёркивает ошибки прямо в коде и помогает отлаживать.

Лекция 3, часть 1. Функции, переменные, условия, циклы

Главная мысль: программа на C — это набор функций, и одна из них обязательно называется `main`. У каждой переменной и функции нужно указать тип.

Функция main — откуда начинается программа

Когда ты запускаешь программу, компьютер ищет функцию `main` и начинает с неё. Это называется **точка входа**. Разберём пример со слайда по строчкам:

```
int main()              // int — main всегда возвращает целое число
{                       // тело функции — внутри фигурных скобок
    // 1. Спрашиваем данные у пользователя
    int start = 0;
    int n_values;
    printf("Enter the number N - number of loops: ");
    scanf("%d", &n_values);      // читаем число с клавиатуры в n_values

    // 2. Основная логика
    int sum = 0;
    for (int i = start; i <= n_values; i++) {
        sum += i;
    }

    // 3. Выводим результат
    printf("sum = %d\n", sum);
    return 0;           // 0 = программа завершилась успешно
}
```

На что обратить внимание:

- `scanf("%d", &n_values)` — значок `&` перед именем обязателен. Он означает «адрес переменной»: мы говорим scanf, куда положить прочитанное число. `%d` — «жду целое число».
- `return 0;` — код завершения. `0` — всё хорошо, `1` — была ошибка, `2` и дальше — ошибки, которые ты придумал сам.
- Хорошая привычка из примера: делить `main` на три части — получить данные, посчитать, вывести.

Если убрать `main`, собрать программу не получится. Ошибка будет на шаге линковки — линкер не нашёл, с чего начинать:

```
Undefined symbols for architecture arm64:
  "_main", referenced from: <initial-undefines>
ld: symbol(s) not found for architecture arm64
```

Как написать свою функцию

Шаблон всегда одинаковый:

```
тип_результата имя_функции(тип аргумента имя_аргумента) {
    ...
    return результат;
}
```

Примеры со слайда:

```
// сложить два целых числа: принимает два int, возвращает int
int sum2nums(int num1, int num2) { return num1 + num2; }

// перемножить два дробных: принимает два double, возвращает double
double mul2nums(double num1, double num2) { return num1 * num2; }
```

И пример посложнее — функция, которая оставляет у числа нужное количество знаков после запятой:

```
float round_float(double num, int dec) {
    // Если нужно 2 знака, считаем 10 в степени 2 = 100
    int number_of_digits = pow(10, dec);

    // Сдвигаем запятую вправо и отрезаем дробную часть
    int num_rounded = (int)(num * number_of_digits);

    // Сдвигаем запятую обратно влево
    return (float)num_rounded / number_of_digits;
}
```

Проследим на числе 1.2345 и dec = 2:

1. `number_of_digits` = 10^2 = 100.
2. 1.2345 × 100 = 123.45. Запись `(int)` превращает его в целое число и просто отбрасывает всё после запятой. Получается 123.
3. 123 / 100 = 1.23. Запись `(float)` нужна, чтобы деление было дробным. Если делить целое на целое, C выбросит дробную часть, и получится 1.

Запись вида `(int)` или `(float)` перед значением называется **приведение типа**: «считай это число целым» или «считай дробным». Мелкий нюанс: это не округление, а отрезание. 1.239 превратится в 1.23, а не в 1.24.

Переменные и константы

Примеры со слайда:

```
int int_value;            // объявили коробку для целого числа
int_value = 1;            // положили в неё 1

const int int_value_const = 1;   // const = константа, менять нельзя
// int_value_const = 2;          // ← так будет ошибка компиляции

bool bool_val = true;     // да/нет (нужен #include <stdbool.h>)
float float_val = 1.1;    // дробное число
double double_val = 1.1;  // дробное число поточнее
char char_val = 'a';      // один символ — в одинарных кавычках
char_val = 'b';           // заменили на 'b'
```

Запомни: один символ пишется в одинарных кавычках `'a'`, а строка — в двойных `"Hello"`. Про строки будет подробно в части 2.

Область видимости: где живёт переменная

Правило очень простое: **переменная существует только внутри тех фигурных скобок `{ }`, где её объявили.** Как только скобка закрылась, переменная исчезает. Как вещи в комнате: вышел из комнаты — их уже не видно.

Задачка со слайда — какие переменные доступны в строке с вопросом?

```
int scope_demo(int arg) {      // arg живёт во всей функции
    int i = 1;                 // i живёт во всей функции

    if (true) {
        int j = 2;             // j живёт только внутри if
        for (;;) {
            int k = 3;         // k живёт только внутри for
        }
    } else {
        int w = 2;             // w живёт только внутри else
    }

    // ??? какие переменные доступны здесь?
    return i;
}
```

Ответ: только `arg` и `i`. Переменные `j`, `k` и `w` остались внутри своих скобок. Про вторую часть этой задачи (с `malloc`) — в части 2.

Условия: if и switch

`if` проверяет условие и выполняет код, если оно истинно:

```
if (условие_1) {
    printf("if\n");         // если верно условие_1
} else if (условие_2) {
    printf("else if\n");    // иначе, если верно условие_2
} else {
    printf("else\n");       // во всех остальных случаях
}
```

Главное правило C: в условии можно написать любое число. Ноль считается «ложь», а всё остальное — «истина». Разберём упражнение со слайда «Где вывод сработает?»:

Код	Напечатается?	Почему
`if (0)`	нет	ноль — это ложь
`if (1)`	да	не ноль
`if (-1)`	да	минус не важен, главное — не ноль
`if (true)`	да	`true` — это просто 1
`if (false)`	нет	`false` — это просто 0
`if (1 || 0)`	да	`||` — «ИЛИ»: хотя бы одно истинно
`if (1 && 0)`	нет	`&&` — «И»: нужно, чтобы оба были истинны
`if (1 ^ 0)`	да	`^` — «исключающее ИЛИ»: истина, если числа разные
`if (1.1)`	да	дробное число, не ноль
`if (0.0)`	нет	ноль
`if ('a')`	да	буква в C — это число (у `'a'` код 97)

switch удобен, когда нужно сравнить одну переменную с несколькими вариантами. Пример со слайда:

```
int color = 1;
switch (color) {
    case 1:                         // если color == 1
        printf("color 1");
        break;                      // break = выйти из switch
    case 2:                         // если color == 2 — ничего не делаем и
идём дальше
    case 3:                         // если color == 2 или 3
        printf("color 2, 3");
        break;
    default:                        // если ничего не подошло
        printf("There is no such color");
        return -1;
}
```

Самая частая ловушка — забытый `break`. switch работает так: находит подходящий `case`, прыгает туда и выполняет всё подряд вниз, пока не встретит `break`. Упражнение со слайда:

```
int color = 2;
switch (color) {
    case 1: { printf("color 1\n"); break; }
    case 2: { printf("color 2\n"); }           // нет break!
    case 3: { printf("color 3\n"); }           // нет break!
    default: { printf("NA\n"); return -1; }
}
```

Ответ: напечатается три строки — `color 2`, `color 3`, `NA`. Программа зашла в `case 2`, не встретила `break` и «провалилась» дальше до самого `return`.

Циклы: for, while, do-while

Цикл повторяет код несколько раз. В C их три:

Цикл	Когда брать	Как работает
`for (int i = 0; i < 10; i++) { }`	знаем, сколько раз повторить	начало; пока условие верно; шаг после каждого повтора
`while (условие) { }`	не знаем, сколько раз	сначала проверяет, потом делает. Может не выполниться ни разу
`do { } while (условие);`	нужно хотя бы один раз	сначала делает, потом проверяет

Задача со слайда про пароль. Есть правильный пароль `"12345"`. Пользователь вводит символы по одному, и нужно остановиться на первой ошибке. Какой цикл выбрать?

Рассуждаем так: чтобы что-то проверить, нужно сначала прочитать хотя бы один символ. «Сначала сделать, потом проверить» — это и есть `do-while`:

```
char pass[] = "12345";
char input_sym;
int i = -1;
do {
    input_sym = getchar();       // прочитать один символ с клавиатуры
    i++;                         // перейти к следующей букве пароля
} while (input_sym == pass[i]);  // продолжать, пока символы совпадают
```

Почему `i` начинается с −1: в первом же проходе он станет 0, и мы сравним первый введённый символ с `pass[0]`. На слайде есть и вариант с `while`, но в нём переменная `input_sym` проверяется до того, как в неё что-то записали. Именно

поэтому тут лучше `do-while`.

Лекция 3, часть 2. Память, строки, библиотеки

Главная мысль: в Python памятью занимается сам язык, а в C — ты. Если просишь память, не забудь её отдать. А компилятор не проверяет, что ты вышел за границу массива.

Три места, где живут данные

Место	Что там лежит	Когда исчезает
Стек	обычные переменные внутри функции: `int x = 5;`	само, когда закрывается `}`
Куча	то, что ты попросил через `malloc`	только когда ты вызовешь `free`
Глобальная область	переменные вне функций и строки в кавычках вроде `"blue"`	когда заканчивается программа

malloc и free: как просить и отдавать память

Обычный массив `int a[5]` имеет фиксированный размер. А если размер станет известен только во время работы? Тогда память просят в куче. Пример со слайда:

```
int* arr = malloc(5 * sizeof(int));      // дай место под 5 int. Внутри
мусор!
int* zero_list = calloc(5, sizeof(int)); // то же самое, но заполнено нулями

arr = realloc(arr, 10 * sizeof(int));    // расширить до 10 элементов

free(arr);         // отдать память обратно
free(zero_list);   // и эту тоже
```

Разберём по кусочкам:

- `sizeof(int)` — сколько байт занимает один `int` (обычно 4). Значит, `5 * sizeof(int)` = 20 байт.
- `malloc` возвращает **адрес** начала выделенного куска. Поэтому переменная — указатель `int*`.

- Дальше с ним можно работать как с обычным массивом: `arr[0] = 7;`, `arr[4] = 9;`.
- Если памяти не хватило, `malloc` вернёт `NULL` («ничего»). Это всегда нужно проверять: `if (arr == NULL) { ... }`.
- **На каждый `malloc` — ровно один `free`.** Забыл `free` — утечка памяти. Сделал два раза — программа упадёт.

Как это выглядит в памяти. `malloc` резервирует ячейки подряд, а `arr` указывает на первую из них:

```
… | arr[0] | arr[1] | arr[2] | arr[3] | arr[4] | …
      ↑                 ↑
     arr             arr + 2
```

Вторая часть задачи про область видимости. Что будет, если выделить память внутри `if`?

```
int scope_demo(int arg) {
    int i = 1;
    if (true) {
        int j = 2;
        int *arr = malloc(10 * sizeof(int));   // выделили 40 байт в куче
    }                                          // ← тут исчезают j и arr
    return i;
}
```

Исчезает переменная `arr` — бумажка с адресом, она жила в стеке. А сами 40 байт в куче остаются занятыми. Адреса больше никто не знает, и вызвать `free` уже нельзя. Это и есть **утечка памяти**. Как потерять ключ от квартиры, которую оплачиваешь.

Строки в C

Строка в C — это просто массив символов, в конце которого стоит специальный символ `'\0'` (нулевой байт). По нему все функции понимают, где строка заканчивается. Слово `"Hello"` в памяти выглядит так:

```
| 'H' | 'e' | 'l' | 'l' | 'o' | '\0' |     ← 6 байт, а не 5
```

На слайде сравниваются два способа создать строку. Выглядят похоже, но работают по-разному:

```
char* str_val = "Hello";        // указатель на строку-константу
str_val[0] = 'h';               // опасно! константу менять нельзя

char str_val_array[] = "Hello"; // собственная копия строки в стеке
str_val_array[0] = 'w';         // можно, стало "wello"
str_val_array[10] = 'w';        // вышли за границу массива (в нём 6 ячеек)
```

Вопрос со слайда: «Тут нет ошибки, почему?»

- `char* str_val = "Hello"` — строка в кавычках лежит в особой области памяти «только для чтения». Компилятор не запретит `str_val[0] = 'h'`, но при запуске программа скорее всего упадёт.
- `char str_val_array[] = "Hello"` — создаётся новый массив из 6 ячеек, и туда копируется строка. Его менять можно.
- `str_val_array[10]` — ячейки № 10 нет. Но **C не проверяет границы массивов**, поэтому компилятор молчит. Буква `'w'` запишется в чужую память.

Что произойдёт, когда пишешь за границу массива, на слайде показано тремя вариантами:

1. **Перезапишешь соседнюю переменную** — «ниндзя-баг». Программа работает, но другая переменная вдруг меняется без причины. Искать такое очень тяжело.
2. **Перезапишешь мусор** — ничего заметного не произойдёт. Тебе просто повезло.
3. **Выйдешь за пределы стека** — программа упадёт с ошибкой.

Задача со слайда «вернуть цвет» (лектор пометил её как важную):

```
char* determineColor(const int color) {
    if (color == 1) {
        return "blue";
    } else if (color == 2) {
        return "green";
    }
    return NULL;          // такого цвета нет
}
```

Почему это работает, если все переменные функции исчезают после `return`? Потому что `"blue"` — это не переменная функции. Строки в кавычках живут в отдельной области всю программу, и мы возвращаем адрес туда. А вот так делать **нельзя**:

```
char* bad() {
    char buf[10] = "blue";   // массив в стеке функции
    return buf;              // ошибка: buf исчезнет, адрес будет вести в
никуда
}
```

Задача со слайда «массив в строку»: превратить `{1, 2, -10, 11}` в строку `"1, 2, -10, 11"`.

```
char* arr2str(const char* arr, size_t size) {
    char* str = malloc(size * 6);  // до 4 символов на число + ", " — с
запасом 6
    if (!str) return NULL;          // память не выделилась

    str[0] = '\0';                  // начинаем с пустой строки
    for (size_t i = 0; i < size; i++) {
        char buf[5];                // до 3 цифр + знак минус + '\0'
        sprintf(buf, "%d", arr[i]); // число → текст
        strcat(str, buf);           // дописать в конец
        if (i < size - 1) strcat(str, ", ");
    }
    return str;                     // тот, кто вызвал, должен потом сделать
free(str)
}

char values[] = {1, 2, -10, 11};
size_t size = sizeof(values) / sizeof(values[0]);   // трюк: длина массива =
4
char* str = arr2str(values, size);
```

Тут три полезных приёма:

- `sizeof(values) / sizeof(values[0])` — размер всего массива, делённый на размер одного элемента, даёт количество элементов. Работает только для настоящего массива, не для указателя.
- `sprintf` — как `printf`, но печатает не на экран, а в строку.
- Память для результата выделяется через `malloc`, чтобы её можно было вернуть из функции (см. задачу про цвет).

Функции для строк

Их не нужно зубрить. Достаточно знать, что они есть, и подглядывать сюда. Все они в `#include <string.h>`, кроме `sprintf` и `sscanf` — те в `<stdio.h>`.

Функция	Что делает	Пример
`strlen`	длина строки (без `\0`)	`strlen("Hello")` → 5
`strcpy`	скопировать строку	`strcpy(dest, "Hi");`
`strcat`	дописать в конец	`strcat(dest, "World");`
`strcmp`	сравнить две строки, 0 = одинаковые	`if (strcmp(s1, s2) == 0)`
`strchr`	найти символ	`strchr(str, 'a')`
`strstr`	найти кусок строки	`strstr(str, "abc")`
`strtok`	разрезать строку по разделителю	`strtok(str, ",")`
`strdup`	сделать копию в куче (потом нужен `free`)	`char* c = strdup(str);`
`sprintf` / `snprintf`	напечатать в строку (вторая следит за размером)	`snprintf(buf, sizeof(buf), "%d", 42);`
`sscanf`	прочитать число из строки	`sscanf("42", "%d", &x);`
`memset` / `memcpy`	заполнить / скопировать кусок памяти	`memset(buf, 0, sizeof(buf));`

Важно: строки нельзя сравнивать через `==`. Так сравнятся адреса, а не буквы. Для сравнения строк всегда нужен `strcmp`.

Библиотеки и файлы .h и .c

Библиотека — готовый набор функций. Чтобы ими пользоваться, нужно написать `#include` в начале файла:

```
#include <stdio.h>     // printf, scanf
#include <stdlib.h>    // malloc, free, atoi
#include <string.h>    // strcmp, strcpy, strlen
#include <math.h>      // pow, abs
#include <stdbool.h>   // bool, true, false
#include "mytools.h"   // твой собственный файл
```

Угловые скобки `< >` — искать в системных библиотеках. Кавычки `" "` — сначала искать в папке твоего проекта.

Код обычно делят на два файла. Представь меню и кухню в ресторане:

- `.h` **(заголовочный файл) — это меню.** В нём только список того, что есть: названия функций, их аргументы, описания структур.
- `.c` **— это кухня.** В нём то, как каждая функция работает внутри.

Пример со слайда — цвет из трёх каналов и функция, которая убирает красный:

```
// ===== color.h — "меню" =====
struct Color {                // структура = несколько переменных под одним
именем
    unsigned char red;        // каждый канал — число от 0 до 255
    unsigned char green;
    unsigned char blue;
};
struct Color* removeRedChannel(struct Color* color);   // только объявление,
без тела
const struct Color COLOR_BLACK = {0, 0, 0};

// ===== color.c — "кухня" =====
struct Color* removeRedChannel(struct Color* color) {
    struct Color* color_copy = malloc(sizeof(struct Color));  // место под
новый цвет
    if (color_copy != NULL) {
        *color_copy = *color;     // скопировать все три канала
        color_copy->red = 0;      // обнулить красный
    }
    return color_copy;
}
```

Запись `color_copy->red` читается «поле `red` у структуры, на которую указывает `color_copy`». Стрелка `->` используется с указателем, а точка `.` — с обычной переменной-структурой: `COLOR_BLACK.red`.

Как сдавать лабы

Перед тем как задать вопрос, поищи ответ в репозитории курса. Вопрос ментору пишется в разделе Discussions на GitHub, а ссылку на него нужно отправить ментору в личку. После проверки будет один из трёх результатов:

- **Change request** — нужно исправить замечания. Лаба уходит в конец очереди.
- **Approve** — предварительно принято, нужно прийти и защитить очно.
- **Merge** — лаба сдана, баллы поставлены. Если ментор сразу сделал merge, на защиту идти не нужно.

Лекция 4. Тесты

Главная мысль: тест — это маленький кусочек кода, который проверяет твою функцию вместо тебя. Ты говоришь: «на входе такое — на выходе должно быть такое», и программа сама проверяет. В лабах тесты нужно будет писать самому.

Три способа проверить функцию

Возьмём калькулятор со слайда. Функция получает два числа и знак и возвращает результат:

```
float calc(short int a, short int b, char symbol) {
    switch (symbol) {
        case '+': return (float)a + b;
        case '-': return (float)a - b;
        case '*': return (float)a * b;
        case '/': return (float)a / b;
        default: return -1;           // неизвестный знак
    }
}
```

Как убедиться, что она работает? На слайде три способа:

1. Через scanf — каждый раз вводишь числа руками и смотришь на ответ. Долго и скучно.

```
short int a, b;
char sym;
printf("Input a, b and symbol: ");
scanf("%hd %hd %c", &a, &b, &sym);
printf("Result is: %f\n", calc(a, b, sym));
```

2. Через запуски — вписываешь числа прямо в код и запускаешь. Удобнее, но проверять всё равно приходится глазами.

```
short int a = 100, b = 2;
char sym = '/';
printf("Result: %f\n", calc(a, b, sym));
```

3. Через тесты — пишешь `assert` с ожидаемым ответом:

```
#include <assert.h>

assert(calc(2, 1, '+') == 3);
assert(calc(2, 1, '-') == 1);
assert(calc(3, 2, '*') == 6);
assert(calc(6, 2, '/') == 3);
```

Как работает `assert`: если условие в скобках верно, ничего не происходит. Если неверно, программа сразу останавливается и пишет, какая строчка не прошла. Запустил один раз — и сразу видишь, всё ли в порядке.

Зачем вообще нужны тесты

- **Быстрее.** Сотня проверок проходит за секунду.
- **Дешевле.** Не нужно каждый раз проверять вручную.
- **Надёжнее.** Тесты ловят ошибки рано. А когда ты что-то меняешь в коде, они сразу покажут, если сломалось то, что раньше работало.
- **Без человеческого фактора.** Программа не устаёт и не пропускает проверки.

Но тесты — не панацея. Если требования меняются каждый день (как в стартапе), тщательные тесты придётся столько же часто переписывать.

Какие бывают тесты

По размеру того, что проверяют (на слайде пирамида):

- **Unit-тесты** — проверяют одну маленькую часть, обычно одну функцию. Их больше всего. Именно такие ты будешь писать.
- **Компонентные** — проверяют блок из нескольких функций. Например, всё, что отвечает за вход в аккаунт.
- **Интеграционные** — проверяют, как разные блоки работают вместе.
- **End-to-End** — проверяют целый сценарий пользователя от начала до конца.

По тому, **что именно** проверяем. Это самое важное для курса:

Вид	Вопрос, на который отвечает	Пример для сортировки
Функциональные	Работает ли в обычных случаях?	`[3, 1, 2]` → `[1, 2, 3]`
Граничные	Не сломается ли на крайних случаях?	пустой массив `[]`, один элемент `[5]`

Вид	Вопрос, на который отвечает	Пример для сортировки
Стресс-тесты	Справится ли с большими или неудобными данными?	миллион случайных чисел; уже отсортированный массив
Тесты на скорость (позже в курсе)	Как растёт время с ростом данных?	замерить время на 1000, 100 000, 10 000 000 элементах

Граничные тесты — самые ценные. Код чаще всего ломается именно на нуле, пустом входе, максимальном числе или `NULL`. Когда придумываешь граничные тесты, спрашивай себя: «а что, если дать ноль? пустоту? очень большое число? отрицательное? что-то совсем не то?»

Хороший тест автоматический, повторяемый (каждый раз даёт одинаковый результат), изолированный (один тест — одна проверка) и понятный.

Пример 1: тесты для калькулятора

Функциональные тесты простые: по одному на каждый знак (+, −, ×, /). А вот как лектор придумал граничные:

- **Границы типа.** Аргументы — `short int`, а он хранит числа от −32768 до 32767. Значит, нужно проверить самые крайние: `32767 + 1`, `-32768 - 1`, `32767 × 32767`.
- **Деление.** На ноль делить нельзя: `5 / 0` должно вернуть ошибку `-1`. А ещё `0 / 5 = 0` и отрицательные: `-10 / 5 = -2`.
- **Неправильный знак.** `'@'` или пустой символ `'\0'` → `-1`.
- **Нули.** `0 + 0`, `0 - 0`, `0 × 0`, `0 / 0`.

И вот как это выглядит в коде (файл тестов со слайда):

```
int main() {
    printf("Functional cases\n");
    assert(calc(2, 1, '+') == 3.0);
    assert(calc(2, 1, '-') == 1.0);
    assert(calc(2, 1, '*') == 2.0);
    assert(calc(2, 1, '/') == 2.0);
    printf("End functional cases\n");

    printf("Border cases\n");
    assert(calc(32767, 1, '+') == 32768.0);
    assert(calc(-32768, 1, '-') == -32769.0);
    assert(calc(32767, 32767, '*') == 1073676289.0);
```

```
    assert(calc(1, 1, '@') == -1);
    assert(calc(1, 1, '\0') == -1);
    printf("End border cases\n");

    printf("Zeros cases\n");
    assert(calc(0, 0, '+') == 0.0);
    assert(calc(0, 0, '-') == 0.0);
    assert(calc(0, 0, '*') == 0.0);
    assert(calc(0, 1, '/') == 0.0);
    assert(calc(0, 0, '/') == -1.);
    assert(calc(1, 0, '/') == -1.);
    printf("End zeros cases\n");
}
```

`printf` между блоками нужны, чтобы видеть, до какого места дошли тесты, если что-то упадёт.

Что нашли тесты. Лектор нашёл в своём калькуляторе две ошибки:

1. Деление на ноль не обрабатывалось. В функцию нужно добавить `if (b == 0) return -1;`.
2. `float` не хватает точности для числа 1 073 676 289. float хранит только около 7 точных цифр, а тут их 10. Про это подробнее в лекции 5.

И ещё одна тонкость, которую стоит заметить: функция возвращает `-1` как ошибку. Но `calc(1, 2, '-')` тоже честно равно `-1`. По результату нельзя понять, была ошибка или нет.

Пример 2: подсчёт делителей числа

Делитель числа n — это число, на которое n делится без остатка. У 6 четыре делителя: 1, 2, 3, 6. Первая версия со слайда делает самое очевидное — перебирает все числа от 1 до n:

```
int countDivisorsOfNumber(int num) {
    int count = 0;
    for (int i = 1; i <= num; i++) {
        if (num % i == 0) count++;    // % — остаток от деления. 0 = делится
нацело
    }
    return count;
}
```

Какие тесты придумал лектор:

Вид	Вход	Ожидаем	Зачем
функциональный	6	4	обычное число (1, 2, 3, 6)
функциональный	5	2	простое число (1, 5)
функциональный	100	9	квадрат (10 × 10)
функциональный	−6	8	отрицательное (±1, ±2, ±3, ±6)
граничный	0	0	ноль
граничный	1	1	самое маленькое
граничный	2 147 483 647	2	самый большой `int`, он простой
стресс	2 147 483 647	меньше 1 секунды	не слишком ли долго

И тесты нашли две проблемы:

- **Слишком медленно.** Для 2 миллиардов цикл делает 2 миллиарда шагов — это около 4 секунд.
- **Отрицательные числа не работают.** Для −6 условие `i <= -6` ложно сразу, и цикл не выполняется ни разу. Получаем 0.

Как ускорить. Делители всегда идут парами. У 100: 1 и 100, 2 и 50, 4 и 25, 5 и 20, 10 и 10. В каждой паре одно число не больше $\sqrt{100} = 10$. Значит, достаточно дойти до корня и на каждый найденный делитель прибавлять 2 (сам делитель и его пару). Исключение — 10 × 10: тут делитель в паре с самим собой, поэтому +1. Вместо 2 миллиардов шагов получается около 46 тысяч.

Код после исправлений со слайда:

```
int countDivisorsOfNumber(int num) {
    size_t limit = sqrt(num >= 0 ? num : -num);   // корень из модуля числа

    int count = 0;
    for (int i = 1; i <= limit; i++) {
        if (num % i == 0) {
            // например, 10 для 100 — делитель, но считается один раз
            count += i * i == num ? 1 : 2;
        }
    }

    if (num < 0) count *= 2;    // у отрицательных ещё делители с минусом
    return count;
}
```

Запись `условие ? A : B` — это короткий `if`: «если условие верно, то A, иначе B».

Тест на время со слайда. `clock()` из `<time.h>` возвращает текущее «время процессора»:

```
clock_t start = clock();                                  // засекли
countDivisorsOfNumber(2147483647);                        // посчитали
double time_spent_s = (double)(clock() - start) / CLOCKS_PER_SEC;   //
сколько секунд прошло
assert(time_spent_s < 1.);                                // должно быть меньше
секунды
```

Тесты для динамического массива (пригодится в лабе)

В лабе ты будешь писать динамический массив `Vector` (что это такое — в лекции 6). У него будут функции создать, добавить, получить, заменить и удалить элемент. Код делится на три файла:

- `vector.h` — список функций («меню»);
- `vector.c` — как они работают («кухня»);
- `tests.c` — тесты. В начале `#include <assert.h>` и `#include "vector.h"`, дальше в `main` блоки `assert`.

Лектор разобрал, какие тесты нужны для каждой функции. Это практически готовый чеклист для лабы:

`vectorCreate(elem_size)` — создать пустой массив

- Функциональные: создать массив для `int`, `float` / `double`, `char`, `bool`, строк `char*` и даже для других векторов `Vector*`.

- Граничные: передать размер элемента 0.

`vectorAppend(vector, el)` — добавить элемент в конец

- Функциональные: добавить один элемент; добавить от 2 до 7 и проверить, что массив вырос; проверить разные типы.
- Граничные: `NULL` вместо массива; `NULL` вместо значения; значение другого типа.

`vectorGet(vector, index)` — получить элемент по номеру

- Функциональные: добавить и сразу получить, сравнить; добавить несколько и проверить все; добавить много (чтобы массив вырос), потом удалить часть и снова проверить.
- Граничные: пустой массив; `NULL`; номер больше, чем элементов; последний элемент после добавления и удаления.

`vectorSet(vector, index, value)` — заменить элемент

- Функциональные: добавить и заменить; заменить все после роста и после уменьшения.
- Граничные: пустой массив; `NULL` вместо массива или значения; отрицательный номер; номер 0; последний номер; номер за пределами.

`vectorRemove` — на слайде остался вопросом для аудитории. Попробуй придумать тесты сам по той же схеме.

Вопрос со слайда для размышления: «А это кстати всё ещё unit-тест?» Чтобы проверить `vectorGet`, нужно сначала вызвать `vectorAppend`. Значит, тест проверяет уже две функции сразу. Строго говоря, это уже не совсем unit-тест.

Лекция 5. Типы, память и указатели

Главная мысль: у каждого типа есть точный размер в байтах. От того, какие типы ты выберешь, зависит, сколько памяти съест программа. Узнать размер любого типа или переменной можно через `sizeof`.

Почему это важно

Пример со слайда: база шахматных партий с Kaggle. Если для всех чисел брать самый большой тип (int64, 8 байт), она занимает **7,21 ГБ**. Если подобрать подходящие типы (например, int16 — 2 байта для рейтинга игрока), то **3,90 ГБ**. Почти вдвое меньше, а данные те же.

А почему C, а не Python? В Python многое спрятано. Например, `x in my_set` выглядит как одно действие. Но лектор показал исходники Python (файл `setobject.c`): внутри всё равно цикл. В C ты пишешь такие циклы сам и видишь, сколько стоит каждое действие. К тому же в Python каждое число — целый объект с дополнительной информацией. Поэтому массив из миллиона целых чисел в Python весит примерно в 5 раз больше, чем в C.

Сколько занимает каждый тип

Сначала простая арифметика. 1 байт = 8 бит. Каждый бит — это 0 или 1. Значит, 8 бит дают $2^8 = 256$ разных комбинаций. Это либо числа от 0 до 255 (если без знака, `unsigned`), либо от −128 до 127 (со знаком). Чем больше байт, тем большие числа влезают.

Тип	Байт	Какие числа влезают
`bool`	1	только 0 или 1
`char`	1	от −128 до 127
`unsigned char`	1	от 0 до 255
`short int`	2	от −32 768 до 32 767
`unsigned short`	2	от 0 до 65 535
`int`	4	от −2 147 483 648 до 2 147 483 647 (примерно ±2 миллиарда)
`unsigned int`	4	от 0 до 4 294 967 295
`long int`, `long long`	8	примерно ±9,2 квинтиллиона
`float`	4	дробные, около 7 точных цифр
`double`	8	дробные, около 15–16 точных цифр
`long double`	16	дробные, ещё точнее
любой указатель	8	адрес в памяти

В названиях вроде `int8_t` и `uint8_t` размер написан прямо в имени: 8 бит = 1 байт. Буква `u` означает «без знака» (unsigned). Размеры `long` и указателей в таблице даны для обычного 64-битного компьютера (Mac, Linux).

Загадки с sizeof со слайдов

Лектор показывал код и спрашивал, что он напечатает. Вот ответы с объяснениями.

bool:

```
sizeof(true)       // 4 байта — неожиданно!
sizeof(bool)       // 1 байт
bool true_ = true;
sizeof(true_)      // 1 байт
bool arr[10];
sizeof(arr)        // 10 байт (10 штук по 1)
bool* arr_ = malloc(10 * sizeof(bool));
sizeof(arr_)       // 8 байт — это размер указателя, а не массива!
```

- Почему `sizeof(true)` = 4? В C слово `true` — это просто замена для числа `1` (тот самый `#define`). А число без типа по умолчанию имеет тип `int`, а это 4 байта.
- Почему `sizeof(arr_)` = 8? `arr_` — это указатель, бумажка с адресом. `sizeof` показывает размер бумажки, а не того, на что она указывает. **Это частая ошибка: узнать длину массива из `malloc` через `sizeof` нельзя.** Длину нужно хранить отдельно.

Целые числа:

```
sizeof(1)             // 4 — обычное число это int
sizeof(2147483647)    // 4 — самое большое число, которое влезает в int
sizeof(2147483648)    // 8 — на единицу больше, в int уже не влезает, стало
long
int arr[10];
sizeof(arr)           // 40 — 10 штук по 4 байта
```

Символы и строки:

```
sizeof('a')          // 4 — в C буква в кавычках считается int!
char char_ = 'a';
sizeof(char_)        // 1 — а переменная типа char — 1 байт
sizeof("a")          // 2 — буква 'a' и завершающий '\0'
sizeof("ы")          // 3 — русская буква занимает 2 байта, плюс '\0'
char str[] = "hello";
sizeof(str)          // 6 — 5 букв + '\0'
char str_k[] = "Привет";
sizeof(str_k)        // 13 — 6 русских букв по 2 байта = 12, плюс '\0'
```

Почему русская буква занимает 2 байта? В один байт влезает всего 256 вариантов, и их хватает только на латиницу, цифры и знаки. Остальные буквы в кодировке UTF-8 занимают 2–4 байта. Отсюда следствие: `strlen("Привет")` вернёт **12, а не 6**. Функции строк считают байты, а не буквы.

Дробные числа: почему они неточные

Дробное число хранится примерно так же, как научная запись в физике: 1,5625 × 2^6. В памяти лежат три части (стандарт IEEE 754):

- **знак** — плюс или минус (1 бит);
- **порядок** (экспонента) — в какую степень возводим 2 (8 бит у float, 11 у double);
- **цифры** (мантисса) — дробная часть (23 бита у float, 52 у double).

Пример со слайда для числа 100: знак «плюс», порядок 6, дробная часть 0,5625. Итого (1 + 0,5625) × 2^6 = 1,5625 × 64 = 100. Тут всё точно.

А вот число 1010,1097 в float превращается в 1010,109680. Цифры показывают не то, что ты записал: 23 битам просто не хватает, чтобы хранить его точно. Это как с 1/3 = 0,3333… — в десятичной записи его точно не записать.

Практический вывод: дробные числа нельзя сравнивать через `==`. Сравнивай с допуском: `fabs(a - b) < 0.000001` («разница очень маленькая»). Именно такая проблема вылезла в тестах калькулятора из лекции 4.

Указатели на пальцах

Представь, что память — это улица с пронумерованными домами. Каждая переменная живёт в каком-то доме. **Указатель** — это бумажка, на которой записан номер дома. Сама бумажка тоже где-то лежит и занимает 8 байт.

Два главных значка:

- `&x` — «адрес x»: узнать номер дома, где живёт `x`.
- `*p` — «то, что лежит по адресу p»: прийти по адресу с бумажки и посмотреть, что внутри. Это называется «разыменование».

Простой пример:

```
int x = 5;
int* p = &x;      // p хранит адрес x
printf("%d", *p); // напечатает 5 — то, что лежит по адресу
*p = 10;          // положили 10 по адресу — теперь x равен 10
```

Теперь пример со слайда:

```
short int val1 = 1, val2 = 2;   // две переменные по 2 байта
short int* var1 = &val1;        // указатель на val1
short int* var2 = &val2;        // указатель на val2

sizeof(var1)    // 8 — указатель всегда 8 байт

printf("%p", (void*)var1);         // 0x16f702e06 — адрес val1
printf("%p", (void*)var2);         // 0x16f702e04 — адрес val2
printf("%p", (void*)(var1 - 1));   // 0x16f702e04 — тот же, что у val2!
```

Что тут интересного:

- `val2` лежит на 2 байта **ниже**, чем `val1` (…e04 меньше, чем …e06). Это потому, что стек заполняется от больших адресов к меньшим.
- `var1 - 1` сдвигает адрес не на 1 байт, а на **один элемент** типа `short` (2 байта). Поэтому мы попадаем ровно в `val2`.

Правило арифметики указателей: `p + 1` сдвигает на размер одного элемента. Для `short*` — на 2 байта, для `int*` — на 4, для `double*` — на 8. Именно поэтому `arr[i]` и `*(arr + i)` — это одно и то же.

И второй пример со слайда — с памятью из кучи:

```
short int* arr = malloc(2 * sizeof(short int));   // место под 2 числа short

printf("%p", (void*)arr);         // 0x15b606070
printf("%p", (void*)(arr + 1));   // 0x15b606072 — на 2 байта дальше

printf("%d", *arr);         // первое число
printf("%d", *(arr + 1));   // второе число
free(arr);
```

В куче, в отличие от стека, элементы идут по возрастанию адресов. На схеме слайда видно, что один `short` — это два байта подряд, которые читаются как одно число. Например, байты со значениями 1 и 2 дают число 2 × 256 + 1 = 513. И помни: память от `malloc` не очищена, там может лежать что угодно.

Структуры и «пустые» байты

Структура (`struct`) — это несколько переменных, собранных под одним именем. Казалось бы, её размер — это сумма размеров полей. Но нет! Задача со слайда:

```
struct A { int x; };              // sizeof = 4
struct B { char c; int x; };      // sizeof = 8, а не 5!
struct C { int x; char c; };      // sizeof = 8, а не 5!
struct D { char c; double d; };   // sizeof = 16, а не 9!
```

Дело в **выравнивании**. Процессор быстрее читает `int`, если тот лежит по адресу, кратному 4. А `double` — если по адресу, кратному 8. Поэтому компилятор вставляет между полями пустые байты. Как в кинотеатре: если ваша компания из четырёх хочет сидеть вместе, иногда рядом остаются пустые кресла.

Раскладка по байтам (`c` — char, `x` — int, `_` — пустой байт, `d` — double):

```
struct B:  [c][_][_][_][x][x][x][x]                                 = 8 байт
struct C:  [x][x][x][x][c][_][_][_]                                 = 8 байт
struct D:  [c][_][_][_][_][_][_][_][d][d][d][d][d][d][d][d]         = 16 байт
```

В `struct C` пустые байты в конце нужны для массива таких структур: тогда `int` в следующей структуре тоже встанет ровно. Совет: объявляй поля от больших к меньшим, так пустого места меньше.

В конце лекции — заготовка к лабе, структура динамического массива. `typedef` позволяет писать просто `Vector` вместо `struct Vector`. Подробно разберём в лекции 6.

```
typedef struct {
    void *data;          // адрес, где лежат сами элементы
    size_t size;         // сколько элементов сейчас
    size_t elem_size;    // сколько байт занимает один элемент
    size_t capacity;     // на сколько элементов выделена память
} Vector;
```

Упражнения со слайда перед 3-й лабой

Лектор советует прорешать их самостоятельно. Отмечай галочками.

Базовые:

- [ ] 1. Объявить `int`, создать указатель на неё, вывести адрес и значение через указатель
- [ ] 2. Поменять местами значения двух переменных через указатели
- [ ] 3. Функция принимает `int*` и увеличивает значение на 1
- [ ] 4. Создать `int[5]`, пройти его указателем и вывести элементы
- [ ] 5. Написать свой `strlen` на указателях

Средние:

- [ ] 6. `swap` для двух элементов массива через указатели
- [ ] 7. Скопировать строку `src` в `dst` через указатели
- [ ] 8. Найти минимум массива через указатель
- [ ] 9. Вывести строку задом наперёд, двигая указатель от конца к началу
- [ ] 10. Создать массив указателей на строки и вывести все строки

Продвинутые:

- [ ] 11. Функция принимает `int**` и выделяет внутри динамический массив («возврат массива»)
- [ ] 12. Создать двумерный массив `int**` динамически, заполнить и вывести
- [ ] 13. Функция принимает массив строк `char**` и сортирует их
- [ ] 14. Свой поиск подстроки (аналог `strstr`) на указателях
- [ ] 15. Структура с полем-указателем: выделить память под поле и освободить её

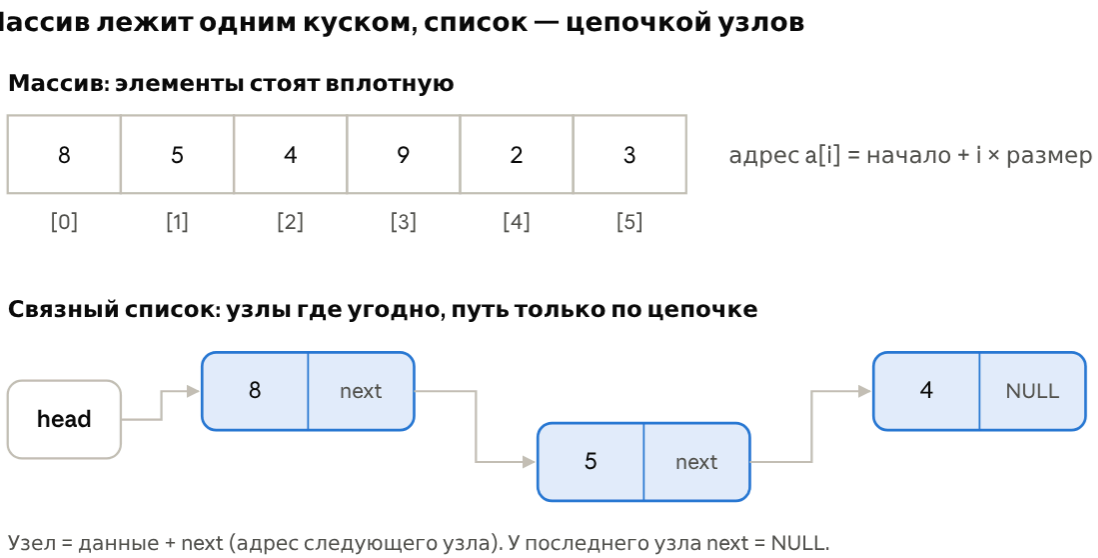
Лекция 6, часть 1. Массив, список и Vector

Главная мысль: данные можно хранить двумя способами — подряд (массив) или цепочкой (связный список). У каждого свои сильные стороны. А чтобы не возиться каждый раз с `malloc` и `realloc` вручную, массив заворачивают в структуру `Vector`, которая сама следит за памятью. Именно её ты будешь писать в лабе.

Массив и связный список: в чём разница

Массив — как вагоны поезда: все стоят вплотную друг за другом. Если знаешь, где первый вагон, любой другой находишь сразу: адрес = начало + номер × размер вагона. Но чтобы вставить вагон в начало, придётся сдвинуть все остальные.

Связный список — как квест с записками. Каждый элемент (узел) может лежать в любом месте памяти. В каждом узле есть данные и записка «следующий лежит там-то» (указатель `next`). Чтобы добраться до пятого элемента, нужно пройти первые четыре. Зато добавить или убрать элемент в начале легко: просто переписать одну записку.



массив и связный список в памяти

Чтобы достать `a[4]` из массива, достаточно одной формулы. Чтобы добраться до третьего узла списка, нужно пройти по стрелкам от `head`.

На слайде два примера задач, где лучше подходит то или другое:

- **Очередь задач** — задачи приходят в конец, а забирают их с обоих концов. Тут лучше **список**: добавлять и убирать с краев быстро.
- **Температура по регионам** (Спб, …, Мск) — нужно быстро получать и часто менять значение по номеру региона. Тут лучше **массив**: доступ по номеру мгновенный.

Сколько стоят операции (асимптотика)

Асимптотика — это ответ на вопрос «во сколько раз дольше станет работать операция, если данных станет больше». Записывают её буквой O:

- **O(1)** — время не зависит от количества данных. 10 элементов или миллион — одинаково быстро.
- **O(N)** — время растёт вместе с количеством данных. В 10 раз больше данных — в 10 раз дольше. Как пройти весь список от начала до конца.
- **Амортизированное O(1)** (на слайде помечено звёздочкой) — почти всегда быстро, но изредка бывает медленная операция. В среднем выходит быстро. Пример ниже.

Таблица со слайда:

Операция	Массив	Односвязный список	Двусвязный список
взять элемент по номеру `[i]`	O(1)	O(N)	O(N)
взять первый или последний	O(1)	O(1)	O(1)
добавить в конец (`append`)	амортизированно O(1)	O(1)	O(1)
заменить по номеру (`set`)	O(1)	O(N)	O(N)
удалить первый `pop(0)`	O(N)	O(1)	O(1)
удалить последний `pop(-1)`	амортизированно O(1)	O(N)	O(1)
удалить из середины `pop(i)`	O(N)	O(N)	O(N)
найти значение (`find`)	O(N)	O(N)	O(N)
сколько занимает памяти	N × размер + служебные поля	N × 16 + 8 байт	N × 24 + 8 байт

Как понять эту таблицу, а не заучить:

- **Массив, `[i]` за O(1):** адрес считается одной формулой, искать не нужно.
- **Массив, `pop(0)` за O(N):** если убрать первый вагон, все остальные придётся сдвинуть на одно место влево.
- **Список, `[i]` за O(N):** нужно идти по запискам от начала.
- **Односвязный список, `pop(-1)` за O(N):** чтобы удалить последний, надо найти предпоследний и стереть у него записку `next`. А записок «назад» нет, поэтому идём от начала. В **двусвязном** списке у каждого узла есть ещё и указатель `prev` на предыдущий, поэтому там O(1).
- **Память:** узел односвязного списка — 16 байт (указатель на данные 8 + указатель `next` 8), двусвязного — 24 (ещё `prev`). Список всегда тяжелее массива.

Динамический массив: size и capacity

Обычный массив в C имеет фиксированный размер. Динамический умеет расти. Внутри него два разных числа, и их важно не путать:

- **`size`** — сколько элементов там сейчас лежит;
- **`capacity`** — на сколько элементов уже выделена память.

Как рюкзак на 10 мест (`capacity = 10`), в котором пока 3 вещи (`size = 3`). Пока есть место, новая вещь просто кладётся в свободную ячейку — это быстро. Когда место кончилось, приходится покупать рюкзак побольше (`realloc`) и перекладывать туда все вещи. Это долго, O(N).

Хитрость в том, чтобы покупать рюкзак **вдвое больше**, а не на одно место. Тогда перекладывать придётся всё реже и реже: на 4-й, 8-й, 16-й, 32-й вещи… В среднем на одно добавление выходит быстро — это и есть «амортизированное O(1)».

Зачем нужен Vector: код без него

В лекции всё показано на одной задаче — сглаживании (полностью она разобрана в части 2). Сначала лектор пишет её на обычных массивах. Вот кусочек, где в массив добавляется очередное число:

```
arr_out[*size_out] = meanValue(window, window_size);   // положили число
*size_out += 1;                                        // увеличили счётчик
arr_out = realloc(arr_out, (*size_out + 1) * sizeof(int8_t));   // расширили
на 1
if (arr_out == NULL) {                                 // проверили, что
получилось
    free(arr_out);
    RETURN_ERROR("Не удалось релоцировать память", NULL);
}
```

Лектор спрашивает: «Что не нравится?» Ответ со слайда:

1. Напрягает выделять память вручную.
2. Напрягает перевыделять память на каждом шаге (и это ещё и медленно — рюкзак растёт на одно место).
3. Напрягает отдельно таскать размер массива в переменной `size_out`.
4. Долго заполняется окно при сглаживании (про это в части 2).

Первые три проблемы решает `Vector`. Мы прячем всю возню с памятью внутрь функций, и снаружи добавление выглядит так:

```
int8_t value = meanValue(window, window_size);
appendVectorItem(arr_out, &value);    // и всё! памятью Vector занимается сам
```

Как устроен Vector

Структура и список функций со слайда (первая версия, только для чисел `int8_t`):

```
typedef struct {
    int8_t* data;      // указатель на блок данных
    size_t size;       // сколько элементов сейчас
    size_t capacity;   // на сколько элементов выделена память
} Vector;

Vector* createVector();                                          // создать
int appendVectorItem(Vector* vector, int8_t* el);                // добавить в
конец
void* getVectorItem(Vector* vector, size_t index);               // получить по
номеру
int setVectorItem(Vector* vector, size_t index, int8_t* value);   // заменить
int8_t* popVectorItem(Vector* vector, size_t index);             // удалить и
вернуть
long int findVectorItem(Vector* vector, int8_t* value);          // найти
int vectorFree(Vector* vector);                                  // освободить
память
```

Лектор советует перед кодом сначала расписать шаги словами. Разберём три функции именно так.

Создание. Шаги: выделить память под саму структуру и проверить; выделить память под данные и проверить; заполнить остальные поля.

```
Vector* createVector() {
    Vector* vector = malloc(sizeof(Vector));   // 1. память под структуру
    if (!vector) return NULL;

    vector->data = malloc(MIN_SIZE * 1);       // 2. память под MIN_SIZE
чисел по 1 байту
    if (!vector->data) {
        free(vector);      // не забываем освободить то, что уже выделили!
        return NULL;
    }

    vector->size = 0;                // 3. пока пусто
    vector->capacity = MIN_SIZE;     //    но место на MIN_SIZE элементов уже
есть
    return vector;
}
```

Обрати внимание на `free(vector)` внутри проверки. Если структуру создать получилось, а данные нет, надо вернуть память под структуру. Иначе утечка.

Добавление. Шаги: проверить, влезает ли новый элемент; если нет, увеличить вместимость; положить значение; увеличить размер.

```
int appendVectorItem(Vector* vector, int8_t* el) {
    if (!vector || !el) return -1;            // защита от NULL
    if (resize(vector, 1) != 0) return -1;    // если нужно, расширить место
    vector->data[vector->size] = *el;         // положить в первую свободную
ячейку
    vector->size++;                           // элементов стало на один
больше
    return 0;                                 // 0 = успех
}
```

Первая свободная ячейка — всегда `data[size]`. Если в массиве 3 элемента, они занимают ячейки 0, 1, 2, а свободная — № 3. (На слайде написано `= el` без звёздочки — это опечатка: так в массив запишется адрес, а не число. Нужно `*el`.)

Удаление. Шаги: проверить номер; сохранить копию удаляемого элемента, чтобы его вернуть; сдвинуть все элементы правее на один влево; уменьшить размер.

```
int8_t* popVectorItem(Vector* vector, size_t index) {
    if (!vector || index >= vector->size) return NULL;   // такого номера нет

    int8_t* copy = malloc(1);                    // место под копию
    if (!copy) return NULL;
    int8_t* item = getVectorItem(vector, index);
    memcpy(copy, item, 1);                       // скопировали 1 байт

    // сдвигаем элементы влево, чтобы закрыть дырку
    for (size_t i = index; i < vector->size - 1; i++) {
        int8_t* src = getVectorItem(vector, i + 1);
        setVectorItem(vector, i, src);
    }

    vector->size--;
    resize(vector, -1);     // если стало слишком пусто, можно уменьшить
память
    return copy;            // тот, кто вызвал, потом делает free(copy)
}
```

Именно цикл сдвига делает удаление из начала медленным (O(N)). Чтобы убрать элемент № 0, нужно передвинуть все остальные.

Серьёзное ограничение этой версии (вопрос со слайда): она работает только с одним типом — `int8_t`. Для `double` или строк пришлось бы писать вторую такую же структуру. Как это исправить — в части 2.

Лекция 6, часть 2. void*, связный список и задача про сглаживание

void* — указатель «на что угодно»

Наш первый `Vector` умел хранить только `int8_t`. Чтобы он мог хранить что угодно, нужен `void*`.

`void*` — это указатель, у которого не сказано, на какой тип он указывает. Как адрес дома без пометки, что там: квартира, магазин или склад. Адрес есть, а что внутри — неизвестно.

```
int x = 10;
void* ptr = &x;   // можно: void* может хранить адрес чего угодно

void* p;
*p = 5;           // ошибка! неизвестно, сколько байт писать
```

Почему ошибка? Чтобы прочитать или записать значение, компилятор должен знать, сколько байт брать: 1 для `char`, 4 для `int`, 8 для `double`. У `void*` этого нет. Поэтому сначала нужно сказать тип — **привести** указатель:

```
int x = 10;
void* p = &x;
int y = *(int*)p;   // (int*)p — «считай, что там int», потом * — «возьми значение»
printf("%d\n", y);  // 10
```

Ты уже пользовался `void*`, сам того не зная: `malloc` возвращает именно его. Поэтому `malloc` подходит для любого типа.

Главный трюк: как найти i-й элемент, не зная типа

Если мы не знаем тип, запись `arr[3]` не работает. Зато мы знаем **размер одного элемента в байтах** (`elem_size`). Тогда адрес i-го элемента можно посчитать вручную: начало + i × elem_size байт. Пример со слайда:

```
const size_t elem_size = sizeof(int);     // 4 байта
int* arr = malloc(10 * elem_size);

// Записать число 4 в ячейку № 3
int value = 4;
memcpy((char*)arr + 3 * elem_size, &value, elem_size);

// Прочитать ячейку № 3
printf("Value: %d\n", *(int*)((char*)arr + 3 * elem_size));
```

Разберём строчку с `memcpy` по кусочкам:

1. `(char*)arr` — временно считаем, что там лежат `char`. Зачем? `char` занимает ровно 1 байт. Тогда `+ 12` сдвинет ровно на 12 байт, а не на 12 элементов (помнишь правило из лекции 5?).
2. `+ 3 * elem_size` — сдвиг до ячейки № 3: 3 × 4 = 12 байт от начала.
3. `&value` — адрес, откуда брать данные.
4. `elem_size` — сколько байт скопировать.

`memcpy(куда, откуда, сколько)` просто копирует байты. Ей всё равно, что это за байты — число, буква или структура. Именно поэтому она подходит для универсального вектора.

Универсальный Vector

Теперь в структуру добавляется поле `elem_size`, а данные хранятся как `void*`:

```
typedef struct {
    void* data;          // данные любого типа
    size_t elem_size;    // размер одного элемента в байтах
    size_t size;         // сколько элементов сейчас
    size_t capacity;     // на сколько элементов выделена память
} Vector;
```

Какой `elem_size` передавать для разных типов (со слайда):

Тип	elem_size
`int`	`sizeof(int)` = 4
`short int`	`sizeof(short int)` = 2
`double`	`sizeof(double)` = 8
`char`	`sizeof(char)` = 1
строка `char*`	пока договорились ограничить 100 байтами
`struct Point`	`sizeof(struct Point)`

И как меняются функции. Везде, где был «1 байт», теперь стоит `elem_size`, а вместо `=` — `memcpy`:

```
Vector* createVector(size_t elem_size) {
    ...
    vector->data = malloc(MIN_SIZE * elem_size);   // было MIN_SIZE * 1
    ...
}

int appendVectorItem(Vector* vector, void* el) {
    ...
    // адрес первой свободной ячейки = начало + size * elem_size
    memcpy((char*)vector->data + vector->size * vector->elem_size, el,
vector->elem_size);
    ...
}

void* popVectorItem(Vector* vector, size_t index) {
    ...
    void* copy = malloc(vector->elem_size);         // было malloc(1)
```

```
    void* item = getVectorItem(vector, index);
    memcpy(copy, item, vector->elem_size);
    ...
}
```

Связный список в коде

Узел списка — это структура из двух полей: данные и адрес следующего узла. Сам список хранит только адрес первого узла («голову», `head`).

```
typedef struct Node {
    void* data;            // данные любого типа
    struct Node* next;     // адрес следующего узла (NULL у последнего)
} Node;

typedef struct {
    Node* head;            // первый узел
    size_t elem_size;      // размер данных в одном узле
} GenericList;

GenericList* createList(size_t elem_size);
void appendItem(GenericList* list, void* data);
unsigned int findItem(GenericList* list, void* value, EqualsFunc cmp);
void* popItem(GenericList* list, size_t index);
unsigned int listLength(GenericList* list);
void freeList(GenericList* list);
```

Внутри `Node` стоит `struct Node* next` — структура указывает на структуру такого же типа. Так и получается цепочка.

Обход списка всегда пишется одинаково. Запомни этот шаблон:

```
Node* current = list->head;     // встаём на первый узел
while (current) {               // пока не дошли до конца (NULL)
    // ... что-то делаем с current->data ...
    current = current->next;    // переходим к следующему
}
```

Указатель на функцию («лямбда» в C)

Задача: найти в списке значение. Но список хранит `void*` и не знает, как сравнивать свои данные. Числа сравниваются через `==`, строки — через `strcmp`. Решение: тот, кто вызывает поиск, сам передаёт функцию сравнения. В C функцию можно передать как аргумент, как обычную переменную. Код со слайда:

```
// Новое имя типа: "функция, которая принимает два void* и возвращает int"
typedef int (*EqualsFunc)(const void* a, const void* b);

// Конкретная функция сравнения для int
static inline int intEquals(const void* a, const void* b) {
    return *(int*)a == *(int*)b;     // привели к int* и сравнили значения
}

unsigned int findItem(GenericList* list, void* value, EqualsFunc cmp) {
    if (!list || !cmp || !list->head) return (unsigned int)-1;

    Node* current = list->head;
    unsigned int index = 0;
    while (current) {
        if (cmp(current->data, value)) return index;   // нашли — вернули
номер
        current = current->next;
        index++;
    }
    return (unsigned int)-1;    // не нашли
}

// Вызов: передаём имя функции без скобок
int x = 5;
findItem(list, &x, intEquals);
```

(На слайде вместо `return index` стоит `break`, и функция всегда возвращает «не найдено». Здесь это исправлено.)

Задача про сглаживание целиком

Условие. Есть файл с числами через пробел, например измерения температуры по часам. Нужно сделать ряд «гладее»: заменить каждую точку средним значением нескольких соседних. Это называется **скользящее среднее**. Результат нужно записать в файл.

Пример с окном 3. Вход: `1 2 3 4 5`. «Окно» из трёх чисел едет слева направо:

```
[1 2 3] 4 5    → среднее 2
1 [2 3 4] 5    → среднее 3
1 2 [3 4 5]    → среднее 4
Выход: 2 3 4
```

Выход короче входа на (размер окна − 1), потому что окно не может вылезти за края.

Лектор предлагает решать такие задачи по плану: расписать шаги словами (псевдокод) → подумать → написать код → подумать → придумать тесты → подумать → написать тесты. «Подумать» здесь значит: что может пойти не так с файлом и с памятью?

Шаг 1. Прочитать числа из файла. Псевдокод со слайда:

```
int8_t* read_array(char* filename) {
    // Создать массив, в который будем читать данные
    // Открыть файл
    // Завести строку для накопления цифр и переменную для символа
    // Прочитать первый символ
    // Пока символ не EOF (конец файла):
    //     Если символ — пробел:
    //         превратить накопленные цифры в число, добавить в массив
    //         очистить строку с цифрами
    //     Иначе:
    //         дописать символ в строку с цифрами
    //     Прочитать следующий символ
    // Закрыть файл
}
```

Идея простая: читаем файл по одной букве. Цифры складываем в маленькую строку-буфер. Когда встретили пробел, число закончилось: превращаем буфер в число и очищаем его. Версия с вектором со слайда, с моими комментариями:

```
Vector* readArrayV2(char* filename) {
    Vector* arr = createVector();
    if (arr == NULL) RETURN_ERROR("Error vector creation", NULL);

    FILE* file = fopen(filename, "r");          // открыть файл на чтение
    if (!file) RETURN_ERROR("Error open file", NULL);

    char digit_str[5] = "";                     // буфер для цифр одного
числа
    int symbol_code = fgetc(file);              // прочитать один символ
    while (symbol_code != EOF) {                // EOF = конец файла
        char symbol = (char)symbol_code;
        if (symbol == ' ') {                    // пробел = число закончилось
            if (strlen(digit_str) > 0) {        // если в буфере что-то есть
                char* end;
                int8_t val = (int8_t)strtol(digit_str, &end, 10);   // текст
→ число
                if (appendVectorItem(arr, &val) == -1) ...
                digit_str[0] = '\0';            // очистить буфер
            }
```

```
        } else {                                    // цифра или минус
            size_t len = strlen(digit_str);
            if (len >= 5) ...                       // число слишком длинное
            digit_str[len] = symbol;                // дописать символ в конец
            digit_str[len + 1] = '\0';              // и закрыть строку
        }
        symbol_code = fgetc(file);                  // следующий символ
    }

    // после последнего числа нет пробела — добавляем его отдельно
    if (strlen(digit_str) > 0) {
        char* end;
        int8_t val = (int8_t)strtol(digit_str, &end, 10);
        if (appendVectorItem(arr, &val) == -1) ...
    }
    fclose(file);                                   // закрыть файл
    return arr;
}
```

Вопросы, которые лектор задавал по этому коду:

- *Почему буфер из 5 символов?* Числа типа `int8_t` — от −128 до 127. Самое длинное — `-128`, это 4 символа, плюс `'\0'` в конце.
- *Что забыли перед `fclose`?* Последнее число. После него в файле нет пробела, и без отдельной проверки оно не попадёт в массив.
- *Где может сломаться `strtol`?* Если в файле буквы (`"a"` превратится в 0 без всякой ошибки) или слишком большое число (`128` не влезает в `int8_t` и превратится в −128).
- *Какую ошибку можно словить, дописывая символ в буфер?* Выход за границу буфера, если в файле длинное число.
- *Как узнать, сколько чисел прочитали?* В первой версии с обычным массивом — никак: указатель не знает длину. Пришлось добавлять аргумент `size_t* arr_size`. А у `Vector` размер всегда лежит внутри, в поле `size`.

Тесты для чтения файла со слайда:

Функциональные	Граничные
положительные: `1 2 3 4 5`	файл не существует
отрицательные: `-1 -2 -3 -4 -5`	пустой файл
смешанные: `-1 1 -2 2 3`	файл из одних пробелов
нули: `0 0 1 0 0`	минимальное число `-128`

Функциональные	Граничные
несколько пробелов подряд	максимальное число `127`
чужие символы: `a 1 2 3`	выход за пределы: `-129` и `128`
числа с нулями впереди: `01 02 03`	число из 5 символов и из 6
	10 000 чисел; 10 000 000 чисел (проверка, что нет утечки памяти)

Шаг 2. Сгладить. Псевдокод: проверить входные данные → создать массив для результата → для каждого положения окна скопировать числа в окно, посчитать среднее, добавить в результат. Версия с вектором:

```
Vector* smoothV2(Vector* arr, const uint8_t window_size) {
    // проверки: окно хотя бы из 2 чисел и меньше всего массива
    if (window_size < 2) RETURN_ERROR("Wrong size", NULL);
    if (window_size >= arr->size) RETURN_ERROR("Wrong size", NULL);

    Vector* arr_out = createVector();
    if (arr_out == NULL) RETURN_ERROR("Error vector creation", NULL);

    int8_t window[window_size];
    for (int i = 0; i < arr->size - window_size + 1; i++) {    // каждое
положение окна
        for (int j = 0; j < window_size; j++) {                 // заполняем
окно
            window[j] = *(int8_t*)getVectorItem(arr, i + j);
        }
        int8_t value = meanValue(window, window_size);         // среднее
        if (appendVectorItem(arr_out, &value) == -1) {          // в
результат
            vectorFree(arr_out);
            RETURN_ERROR("Не удалось релоцировать память", NULL);
        }
    }
    return arr_out;
}
```

Лектора здесь «бесит» вложенный цикл. На каждом шаге окно заполняется заново, хотя поменялось всего два числа: одно ушло слева, одно пришло справа.

Шаг 3. Окно из связного списка. Окно — это очередь: новое число встаёт в конец, старое уходит из начала. Для очереди список идеален: добавить в конец и убрать из начала — оба за O(1). У массива удаление из начала стоило бы O(N).

```
Vector* smoothV3(Vector* arr, const uint8_t window_size) {
    ... // те же проверки и создание arr_out

    GenericList* list = createList(arr->elem_size);       // окно-очередь
    for (int i = 0; i < arr->size; i++) {
        appendItem(list, getVectorItem(arr, i));          // новое число — в
конец окна

        if (listLength(list) > window_size)               // окно стало
слишком длинным?
            free(popItem(list, 0));                       // старое число —
из начала окна

        if (listLength(list) < window_size) continue;     // окно ещё не
набралось

        int8_t value = meanListValue(list);               // среднее по окну
        if (appendVectorItem(arr_out, &value) == -1) { ... }
    }
    return arr_out;
}
```

Я немного поправил порядок проверок. На слайде лишнее число удаляется слишком рано, и в окне оказывается на одно число меньше, чем нужно.

Идея, как ускорить ещё (на лекции её не было, но полезно знать): храни сумму чисел в окне. Когда окно сдвигается, прибавляй новое число и вычитай ушедшее. Тогда среднее считается сразу, без прохода по всему окну.

Ошибки в коде со слайдов и вопросы для самопроверки

Код на слайдах показывает идею, но копировать его в лабу как есть нельзя. В нём есть ошибки. Часть лектор оставил нарочно, с подписью «Остановиться и подумать!», а часть — опечатки. Их полезно знать: именно такие ошибки менторы ищут при проверке лаб.

1. Перепутана проверка пробела (лекция 6, чтение файла).

```
if (strlen(digit_str) == ' ')   // плохо: длина строки сравнивается с
пробелом (его код 32)
if (symbol == ' ')              // правильно: сравниваем прочитанный символ
```

2. **Увеличивается не то** (там же).

```
*arr_size++;     // плохо: ++ срабатывает раньше *, и сдвигается сам
указатель
(*arr_size)++;   // правильно: скобки говорят "сначала возьми значение, потом
увеличь"
```

3. **Утечка при неудачном realloc** (чтение файла и первая версия сглаживания).

```
// плохо:
arr = realloc(arr, new_size);
if (arr == NULL) { free(arr); ... }
```

Если `realloc` не смог выделить память, он вернёт `NULL`. Но старый блок при этом остаётся жив. Мы сразу записали `NULL` в `arr` и потеряли адрес старого блока. `free(NULL)` ничего не делает. Правильно — через временную переменную:

```
int8_t* tmp = realloc(arr, new_size);
if (tmp == NULL) { free(arr); return NULL; }   // старый адрес всё ещё в arr
arr = tmp;
```

4. **Утечка, если файл не открылся** (чтение файла). Сначала создаётся массив, потом открывается файл. Если файла нет, функция выходит, а массив так и не освобождён. Решение: сначала открыть файл, потом выделять память. Или сделать `free` перед выходом.

5. **Выход за границу буфера** (чтение файла). В буфере `char digit_str[5]` ячейки с номерами 0–4. Проверка `len >= 5` срабатывает слишком поздно. При `len = 4` код пишет символ в ячейку 4, а `'\0'` — в ячейку 5, которой нет. Правильная проверка: `if (len + 1 >= sizeof(digit_str))`.

6. **В массив записывается адрес вместо числа** (`appendVectorItem` и `smoothV2`).

```
vector->data[vector->size] = el;          // плохо: el — это адрес
vector->data[vector->size] = *el;         // правильно: *el — число по этому
адресу

window[j] = getVectorItem(arr, i + j);             // плохо: функция
возвращает указатель
window[j] = *(int8_t*)getVectorItem(arr, i + j);   // правильно
```

7. **Поиск всегда ничего не находит** (`findItem`). После `break` функция всё равно доходит до `return -1`. Нужно `return index;` прямо внутри `if`.

8. В окне на одно число меньше (`smoothV3`). Старое число удаляется до подсчёта среднего, когда окно только набралось. Правильный порядок — в разделе про лекцию 6, часть 2.

9. Делители отрицательных квадратов (лекция 4). Для `num = -100` проверка `i * i == num` никогда не сработает (10 × 10 = 100, а не −100). В итоге получится 20 делителей вместо 18. Нужно сравнивать с модулем: `i * i == abs(num)` .

10. while в задаче про пароль (лекция 3). Переменная `input_sym` проверяется до того, как в неё что-то записали. Там лежит мусор. Лучше вариант с `do-while` .

Вопросы для самопроверки

Попробуй ответить сам, а потом проверь себя по ответу после тире.

1. Чем компилятор отличается от интерпретатора? — Компилятор переводит всю программу заранее в готовый файл, интерпретатор выполняет код строчка за строчкой.
2. Какие 4 шага делает gcc? — Препроцессинг (`.i`), компиляция (`.s`), ассемблирование (`.o`), линковка (готовая программа).
3. На каком шаге будет ошибка, если забыть `main` ? — На линковке.
4. Что делает `#define LAST 10` ? — Заменяет все `LAST` в тексте на `10` ещё до компиляции.
5. Что значит `return 0` в `main` ? — Программа завершилась успешно.
6. Выполнится ли `if (-1)` ? — Да, любое число кроме нуля — истина.
7. Что будет в switch без `break` ? — Выполнятся все case ниже подходящего, пока не встретится `break` или `return` .
8. Чем `while` отличается от `do-while` ? — `while` сначала проверяет, `do-while` сначала делает и выполняется хотя бы один раз.
9. Где живёт переменная, объявленная внутри `if { }` ? — Только внутри этих скобок.
10. Что такое утечка памяти? — Память выделили через `malloc` , а адрес потеряли и освободить уже не можем.
11. Чем `malloc` отличается от `calloc` ? — `calloc` заполняет память нулями, а в `malloc` мусор.
12. Почему можно вернуть `"blue"` из функции, а локальный `char buf[10]` нельзя? — Строка в кавычках живёт всю программу, а локальный массив исчезает вместе с функцией.
13. Сколько байт занимает строка `"hello"` ? — 6: пять букв и `'\0'` .
14. Как правильно сравнить две строки? — `strcmp(a, b) == 0` , а не `a == b` .

15. Чем функциональный тест отличается от граничного? — Функциональный проверяет обычные случаи, граничный — крайние: ноль, пустоту, максимум, `NULL`.
16. Что делает `assert(x == 3)`? — Если x не равен 3, программа останавливается и пишет, где ошибка.
17. Чему равно `sizeof('a')` в C? — 4, потому что буква в кавычках считается `int`.
18. Чему равно `sizeof` указателя на массив из 100 чисел? — 8. Это размер адреса, а не массива.
19. Почему `struct { char c; int x; }` занимает 8 байт, а не 5? — Из-за выравнивания: после `char` стоят 3 пустых байта, чтобы `int` начинался с адреса, кратного 4.
20. Почему дробные числа нельзя сравнивать через `==`? — Они хранятся приближённо, поэтому сравнивают с допуском.
21. На сколько байт сдвинется `p + 1`, если `p` — это `int*`? — На 4, на размер одного `int`.
22. Чем `size` отличается от `capacity`? — Сколько элементов есть сейчас и на сколько выделена память.
23. Почему удалить первый элемент массива долго, а списка — быстро? — В массиве придётся сдвинуть все остальные элементы, а в списке достаточно переназначить `head`.
24. Почему `void*` нельзя разыменовать через `*`? — Компилятор не знает, сколько байт читать. Сначала нужно привести тип: `*(int*)p`.
25. Зачем писать `(char*)` в `(char*)data + i * elem_size`? — Чтобы сдвиг считался в байтах: `char` занимает ровно 1 байт.